ISPTEC — INSTITUTO SUPERIOR POLITÉCNICO DE TECNOLOGIAS E CIÊNCIAS

Computação Científica II — Resolução passo a passo

Ficha de Trabalho · Estruturas de repetição (for, while e do-while) em C

DISCIPLINA	DOCENTE	SEMESTRE	LINGUAGEM
CC II – Computação Científica II	André Filemon	1.º	C

Conceitos usados e significado dos símbolos

Estrutura / operador	Para que serve
`for (inicio; condicao; passo)`	Repetição com **número de voltas conhecido**
`while (condicao)`	Repete **enquanto** a condição for verdadeira (testa antes)
`do { ... } while (condicao);`	Repete **pelo menos uma vez** (testa depois) — ideal para validar dados
`if (...) { ... } else { ... }`	Decisão
`%` (resto da divisão)	`n % 2 == 0` → n é par; `n % d == 0` → d divide n
`+=`, `*=`, `++`	`x += y` é `x = x + y`; `x *= y` é `x = x * y`; `i++` soma 1 a `i`
`&&`, `||`	«E» lógico, «OU» lógico
`scanf` / `printf`	Ler do teclado / escrever no ecrã

Significado dos tipos e formatos

`int` — número inteiro · `long long` — inteiro grande · `double` — número real · `char` — um carácter

`%d` — inteiro · `%lf` — `double` na leitura · `%.2f` — real com 2 casas decimais · `%c` — carácter · `%lld` — `long long`

`#define N 20` — constante simbólica (o compilador substitui `N` por 20)

acumulador — variável que soma valores a cada volta (`soma += salario`)

contador — variável que conta ocorrências (`mulheres++`)

divisor — d é divisor de n se o resto de n/d é zero

$\bar{x} = \dfrac{\sum x_i}{N}$ — média aritmética de N valores

Exercício 1 — Pesquisa entre habitantes da província de Luanda

> Foram coletados os dados de idade, sexo (F/M) e salário de **20 pessoas**. Escrever um programa em C que mostre: a) a média de salário do grupo; b) maior e menor idade do grupo; c) quantidade de mulheres com salários até AKZ 100 000.

Passo 1 — Entradas e saídas. Entradas: idade (`int`), sexo (`char`) e salário (`double`) de cada pessoa. Saídas: média, maior/menor idade e contagem.

Passo 2 — Variáveis: `soma` (acumulador de salários), `maior` e `menor` (idades extremas), `mulheres` (contador).

Passo 3 — Ciclo `for` de 1 a 20 (número de pessoas conhecido):

- lê os três dados;
- soma o salário a `soma`;
- na 1.ª pessoa, `maior` e `menor` recebem a idade lida; nas seguintes compara-se (`idade > maior`, `idade < menor`);
- se for mulher **e** o salário for ≤ 100 000, incrementa `mulheres`.

Passo 4 — Depois do ciclo: média = `soma / N`.

```
#include <stdio.h>

#define N 20                /* numero de pessoas pesquisadas */
#define LIMITE_SALARIO 100000.0

int main(void)
{
    int i, idade, maior, menor, mulheres = 0;
    char sexo;
    double salario, soma = 0.0;

    for (i = 1; i <= N; i++) {
        printf("Pessoa %d - idade, sexo (F/M) e salario (AKZ): ", i);
        scanf("%d %c %lf", &idade, &sexo, &salario);

        soma += salario;                    /* acumula salarios       */

        if (i == 1) {                       /* 1.a pessoa inicia      */
            maior = idade;                  /* maior e menor idade    */
            menor = idade;
        } else {
            if (idade > maior) maior = idade;
            if (idade < menor) menor = idade;
        }

        if ((sexo == 'F' || sexo == 'f') && salario <= LIMITE_SALARIO)
            mulheres++;                     /* conta as mulheres      */
    }

    printf("\na) Media de salario do grupo: %.2f AKZ\n", soma / N);
    printf("b) Maior idade: %d anos | Menor idade: %d anos\n", maior, menor);
    printf("c) Mulheres com salario ate 100000 AKZ: %d\n", mulheres);
    return 0;
}
```

Teste (dados de exemplo): média $196\,850{,}00$ AKZ; maior idade 58; menor idade 18; 3 mulheres com salário ≤ 100 000 — igual ao valor calculado independentemente.

Exercício 2 — Números ímpares entre 100 e 400

> Fazer um programa em C que gera e mostre os números ímpares entre 100 e 400.

Ideia: o 1.º ímpar depois de 100 é **101**; depois soma-se **2** a cada volta, parando antes de 400 (o último é **399**). Assim não é preciso testar se cada número é ímpar.

```c
#include <stdio.h>

int main(void)
{
    int n;

    printf("Numeros impares entre 100 e 400:\n");
    for (n = 101; n < 400; n += 2)     /* 101, 103, ..., 399 */
        printf("%d ", n);
    printf("\n");
    return 0;
}
```

Alternativa (testando cada número): `for (n = 100; n ⩽ 400; n++) if (n % 2 ≠ 0) printf("%d ", n);`

Resultado: 101 103 105 … 397 399 — são **150** números ímpares, de 101 a 399.

Exercício 3 — Pares entre a e b (5 pares de valores)

> Escrever um programa em C que lê 5 pares de valores a, b, todos inteiros e positivos, um par de cada vez, e com $a < b$, mostre os inteiros pares de a até b, incluindo o a e o b se forem pares.

Passo 1 — Ciclo exterior `for` repete 5 vezes (um par por volta).

Passo 2 — Validação com `do-while`**:** pede os valores **pelo menos uma vez** e repete enquanto forem inválidos (`a ⩽ 0 || b ⩽ 0 || a ⩾ b`).

Passo 3 — Ciclo interior `for` de $n = a$ até $n = b$ (inclui os dois extremos, por causa do `⩽`): imprime n se `n % 2 == 0`.

```c
#include <stdio.h>

int main(void)
{
    int par, a, b, n;

    for (par = 1; par ⩽ 5; par++) {
        do {                                  /* valida a entrada       */
            printf("Par %d - introduza a e b (inteiros positivos, a < b): ", par);
            scanf("%d %d", &a, &b);
        } while (a ⩽ 0 || b ⩽ 0 || a ⩾ b);

        printf("Pares de %d ate %d: ", a, b);
        for (n = a; n ⩽ b; n++)
            if (n % 2 == 0)                   /* resto 0 → par          */
                printf("%d ", n);
        printf("\n");
    }
    return 0;
}
```

Teste (entradas 2 9, 3 3 × repete, 5 10, 1 4, 10 15, 7 12):

Par	Saída
2 9	2 4 6 8
5 10	6 8 10
1 4	2 4
10 15	10 12 14
7 12	8 10 12

O par «3 3» foi rejeitado e voltou a pedir (pois exige $a < b$).

Exercício 4 — Produto dos números primos entre 1 e 20

> Elabore um programa em C que calcule e mostre o produto dos números primos entre 1 e 20. (Número primo é todo número que possui apenas dois divisores, o número 1 e o próprio número.)

Passo 1 — Usar a definição do enunciado: n é primo se tiver **exactamente 2 divisores** (1 e n). O número 1 só tem 1 divisor, por isso não é primo.

Passo 2 — Ciclo exterior de $n = 1$ a 20. Para cada n:

- ciclo interior $d = 1, \ldots, n$ conta os divisores (`n % d == 0`);
- se `divisores == 2`, mostra n e multiplica-o ao acumulador `produto` (inicializado a **1**, o elemento neutro do produto).

Passo 3 — Tipo da variável: o produto é grande, por isso usa-se `long long`.

```c
#include <stdio.h>

int main(void)
{
    int n, d, divisores;
    long long produto = 1;

    printf("Primos entre 1 e 20: ");
    for (n = 1; n ⩽ 20; n++) {
        divisores = 0;
        for (d = 1; d ⩽ n; d++)          /* conta os divisores de n */
            if (n % d == 0)
                divisores++;

        if (divisores == 2) {            /* primo: so 1 e n         */
            printf("%d ", n);
            produto *= n;
        }
    }
    printf("\nProduto dos primos = %lld\n", produto);
    return 0;
}
```

Cálculo à mão (confirma o programa): primos entre 1 e 20: $2, 3, 5, 7, 11, 13, 17, 19$.

$$2 \times 3 \times 5 \times 7 \times 11 \times 13 \times 17 \times 19 = 9\,699\,690$$

Resposta: os primos são 2, 3, 5, 7, 11, 13, 17, 19 e o produto é **9 699 690**.

Como compilar e executar: `gcc ex1_pesquisa_luanda.c -o ex1` e depois `./ex1` (Linux/Mac) ou `ex1.exe` (Windows). Os ficheiros `.c` estão também na pasta `codigo-C` do repositório. Os programas foram compilados com `-Wall -Wextra` sem avisos e testados.